SIMATS ENGINEERING

ASSESSMENT TOOL 2

Simulation Task

COURSE NAME: SIGNALS AND SYSTEMS **COURSE CODE: ECA0302**

COURSE OUTCOME COVERED

CO3: Analyze continuous-time LTI systems using differential equation modeling, convolution, and transform methods, and evaluate system behavior in terms of stability and frequency response for engineering applications.

Assessment Tool Number `: CO3_AT2
Assessment Tool Weightage : 35%
Assessment Tool : Simulation Task
Duration : 60 minutes
Marks : 50
Type of Assessment Conduction : Hybrid
Evaluation Type : Online Evaluation(Github)

Topics Covered in this assessment: System modeling using differential equations , zero-input and zero-state responses, impulse response, frequency response, region of convergence (ROC), stability analysis, Nyquist plots for power grid stability, convolution for impulse response testing, LTI system analysis using Fourier and Laplace transforms, ordinary differential equation modeling for automotive suspension systems.

Description:

- **The simulation task is conducted as home assessment assesses students' ability to implement and analyze signals and systems concepts using tools such as MATLAB and Scilab.**
- **Students develop programs to model continuous-time and discrete-time systems, perform operations like convolution, and analyze frequency responses.**
- **The activity emphasizes visualization of signals, system behavior, and validation of theoretical results through simulation. Students interpret outputs such as plots and spectra to draw meaningful conclusions.**
- **This assessment enhances practical skills, computational understanding, and the ability to apply theory using simulation tools.**

QUESTIONS: **Total Marks:50**

S.No	Question	Bloom's Level	Marks
18	**Gaussian input pulse X(f) = exp(-(pi*f*sigma)^2), sigma=5e-12 s. Channel H(f)=exp(-j*beta2*(2*pi*f)^2*L/2), beta2=-20e-24, L=50e3 m. In Scilab compute Y(f)=X(f).*H(f) over f vector. Take IFFT to recover y(t). Plot input and output pulses. Calculate output pulse width (RMS) and compare with analytical broadening formula.**	**BT4**	**10**

Code of Conduct:

*I **M.Venkata Sai Charan Teja** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

M.v.s.charan teja
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	**25%**	**Develops accurate, well-structured simulation/model independently**	**Develops correct model with minor errors**	**Model is incomplete or requires guidance**	**Unable to develop a proper model**
Tool Usage	**20%**	**Uses simulation tools effectively with correct setup and execution**	**Uses tools with minor errors or guidance**	**Limited ability to use tools properly**	**Unable to use simulation tools**
Analysis of Results	**20%**	**Provides clear, logical analysis and meaningful interpretation of results**	**Analysis is reasonable with minor gaps**	**Superficial analysis; limited interpretation**	**Unable to analyze or interpret results**
Documentation	**15%**	**Presents results clearly with proper documentation and structured reporting**	**Documentation is adequate with minor issues**	**Poorly organized or incomplete documentation**	**No proper documentation or presentation**

Code :

```
Clc;
Clear;
Clf;

// Parameters
Sigma = 5e-12;
Beta2 = -20e-24;
L = 50e3;

// Frequency grid
N = 2^14;
Fmax = 2e11;
Df = 2*Fmax/N;
F = (-N/2:N/2-1)*df;

// Input spectrum X(f)
Xf = exp(-( %pi * f * sigma).^2);

// Channel H(f)
Hf = exp(-%i * beta2 * (2*%pi*f).^2 * L / 2);
```

```
// Output spectrum
Yf = Xf .* Hf;

// Time grid
Dt = 1/(N*df);
T = (-N/2:N/2-1)*dt;

// Input pulse in time domain
Xt = fftshift(ifft(ifftshift(Xf)));

// Output pulse in time domain
Yt = fftshift(ifft(ifftshift(Yf)));

// Normalize
Xt = xt / max(abs(xt));
Yt = yt / max(abs(yt));

// Intensity
Ix = abs(xt).^2;
Iy = abs(yt).^2;

// RMS pulse width function
Function s = rms_width(t,I)
   I = I / sum(I);
   T0 = sum(t .* I);
   S = sqrt(sum(((t – t0).^2) .* I));
Endfunction

Sigma_in = rms_width(t,Ix);
Sigma_out = rms_width(t,Iy);

// Analytical broadening
Sigma_analytical = sigma * sqrt(1 + (beta2*L/sigma^2)^2);

// Display results
Disp(“Input RMS width (s)      = “ + string(sigma_in));
Disp(“Output RMS width (s)     = “ + string(sigma_out));
Disp(“Analytical RMS width (s) = “ + string(sigma_analytical));
Disp(“Broadening factor (sim)  = “ + string(sigma_out/sigma_in));
Disp(“Broadening factor (theory)= “ + string(sigma_analytical/sigma));

// Plot frequency domain
Scf(1);
Subplot(2,1,1);
Plot(f/1e9, abs(Xf));
```

```
Xlabel(“Frequency (GHz)”);
Ylabel(“|X(f)|”);
Title(“Input Gaussian Spectrum”);
Xgrid();

Subplot(2,1,2);
Plot(f/1e9, abs(Yf));
Xlabel(“Frequency (GHz)”);
Ylabel(“|Y(f)|”);
Title(“Output Spectrum after Dispersion”);
Xgrid();

// Plot time domain pulses
Scf(2);
Plot(t*1e12, Ix, ‘b’);
Plot(t*1e12, Iy, ‘r’);
Xlabel(“Time (ps)”);
Ylabel(“Normalized Intensity”);
Title(“Input and Output Pulses”);
Legend([“Input”,”Output”]);
Xgrid();
```

Simulation output: